

Introducción a los Lenguajes de Programación en Ciencias PYTHON EN CIENCIA DE DATOS

Henry R. Moncada

Universidad Nacional del Callao
Facultad de Ciencias Naturales y Matemáticas

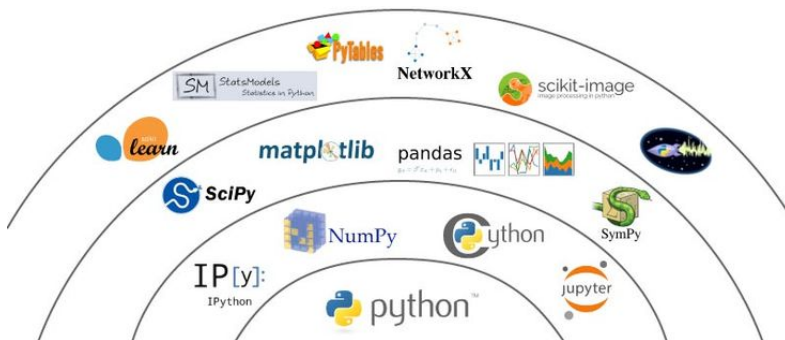
13 de diciembre de 2025

- 1 Introducción a Python en Ciencia de Datos
- 2 Bibliotecas especializadas
- 3 Análisis de Datos con Pandas
- 4 Introducción a DataFrames
 - Importar Archivo
 - Limpiar Datos
 - Ordenar y Agregar Datos
 - Exportar Datos
 - Ejemplos de Análisis de Datos con Pandas
 - Las ventajas de usar Pandas?
- 5 Visualización de Datos
 - Introducción a Matplotlib
 - Introducción a Seaborn
- 6 Machine Learning
 - Machine Learning con Scikit-learn
 - Visualización de Datos con Matplotlib
 - Evaluación de Modelos con Scikit-learn
- 7 Consideraciones y Recomendaciones
- 8 Conclusión

¿Por qué Python?

Python es uno de los **lenguajes más populares en ciencia de datos** debido a:

- Su **sintaxis** clara y legible.
- Una amplia gama de **bibliotecas especializadas** (NumPy, Scipy, Pandas, Matplotlib, Scikit-learn).
- **Comunidad activa** y soporte continuo.
- **Versatilidad en aplicaciones** (análisis de datos, aprendizaje automático, visualización, etc.).



Hola mundo en Python

- Python 2.7 fue lanzado en 2010 como última versión mayor de la serie 2.x.
- Python 3 introdujo cambios incompatibles con 2.x.
- Python 2 llegó al fin de su vida útil en enero de 2020.

```
(base) lola:~$ vi 0.Hello_World.py
```

```
1  ## 1.Print to the terminal
2  # python Hello_World.py
3
4  message = "Hello World!!!"
5  print(message)
6
7  print("Buenas Noches")
8
9  # Perform addition operation 1+1 and
   print the result
10 result = 1 + 1
11 print(result)
12 print("suma = %d"% result)
```

Python programa: Hola Mundo

- **print** es una función en Python 3.
- **Division:** $3/2$ es distinto en Python 2 que en 3.
- Unicode por defecto en Python 3.
- Cambios en funciones como **range()**, **input()**, **dict.keys()**.

Executar programa

```
(base) lola:~$ python 0.Hello_World.py
```

```
Hello World!!!
```

```
Buenas Noches
```

```
2
```

```
suma = 2
```

```
2 103      Jana      Prague   33      81.0
```

Python como Calculadora

- Abrir un terminal CTRL+ALT+T
- Escribir **python** despues de prompt

```
1 (base) lola:~$ python
2 Python 3.11.7 (main, Dec 15 2023, 18:12:31) [GCC 11.2.0] on linux
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>>
```

```
1 (base) lola:~$ python
2 Python 3.11.7 (main, Dec 15 2023, 18:12:31) [GCC 11.2.0] on linux
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>> a=2
5 >>> b=3
6 >>> c=a+b
7 >>> c
8 5
9 >>> c**2
10 25
```

- Importar librerias

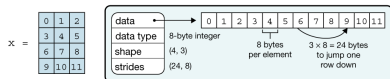
```
1 (base) lola:~$ python
2 Python 3.11.7 (main, Dec 15 2023, 18:12:31) [GCC 11.2.0] on linux
3 Type "help", "copyright", "credits" or "license" for more information.
4 >>> import numpy as np
5 >>> np.sqrt(25)
6 5.0
```

1. NumPy

- Soporte para arrays multidimensionales.
- Operaciones vectorizadas de alto rendimiento.
- Base para muchas otras bibliotecas.



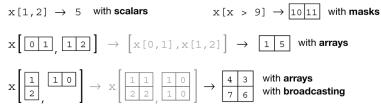
a Data structure



b Indexing (view)



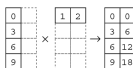
c Indexing (copy)



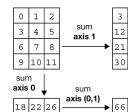
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
In [2]: x = np.arange(12)
In [3]: x = x.reshape(4, 3)

In [4]: x
Out[4]:
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])

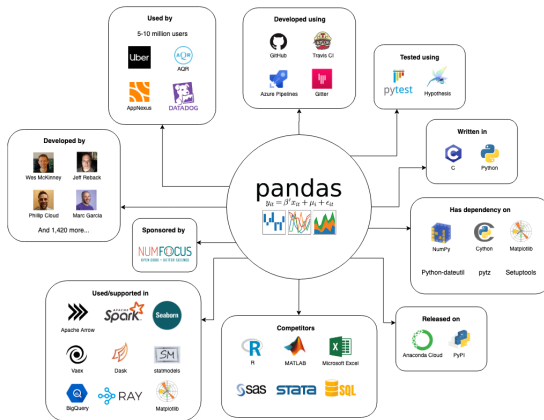
In [5]: np.mean(x, axis=0)
Out[5]: array([4.5, 5.5, 6.5])

In [6]: x = x - np.mean(x, axis=0)

In [7]: x
Out[7]:
array([[-4.5, -4.5, -4.5],
       [-1.5, -1.5, -1.5],
       [ 1.5,  1.5,  1.5],
       [ 4.5,  4.5,  4.5]])
```

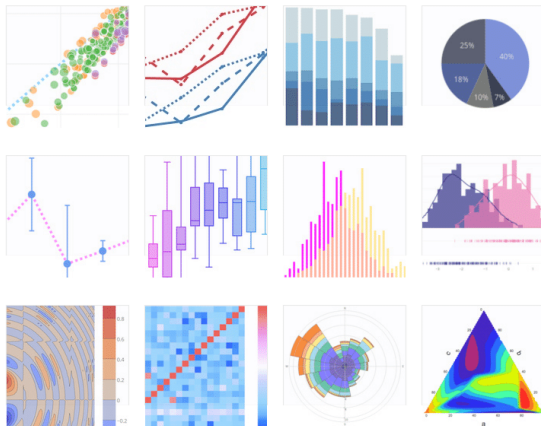
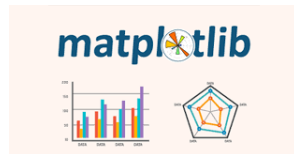
2. Pandas

- Estructuras de datos: **DataFrame** y **Series**.
- Limpieza, transformación y análisis de datos.
- Ideal para datos tabulares.



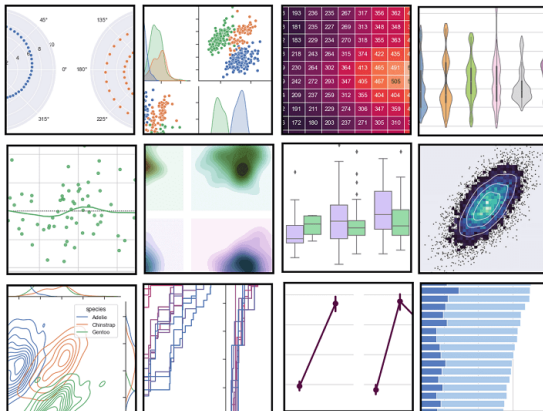
3. Matplotlib

- Biblioteca de visualización 2D.
- Gráficos de líneas, barras, histogramas, dispersión, etc.
- Altamente personalizable.



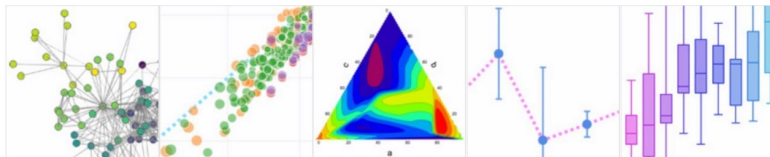
4. Seaborn

- Basada en Matplotlib, pero con una interfaz de alto nivel.
- Visualizaciones estadísticas elegantes.
- Gráficos como mapas de calor, diagramas de violín, etc.

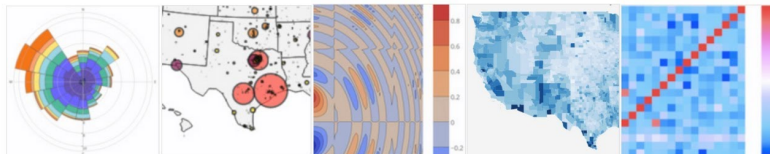


5. Plotly

- Visualización interactiva en web.
- Gráficos 3D, dashboards y mapas.
- Compatible con Jupyter Notebooks.



LEARN PLOTLY



6. SciPy

- Extiende NumPy con módulos para:

- Álgebra lineal
- Optimización
- Estadística
- Señales y más

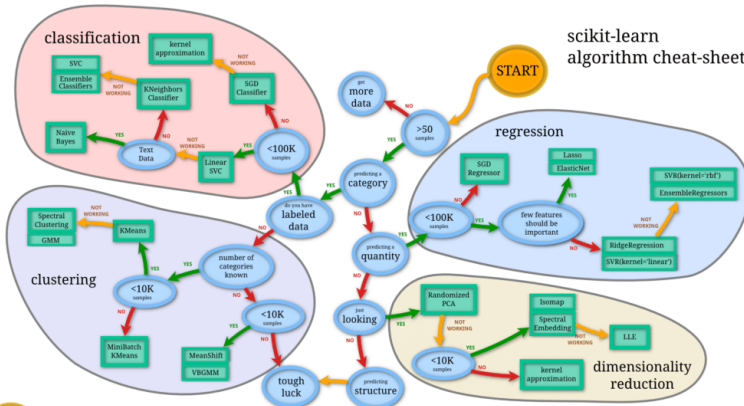


7. Scikit-learn

- Biblioteca de Machine Learning.
- Modelos de regresión, clasificación, clustering y reducción de dimensionalidad.
- Fácil de usar, ideal para prototipos rápidos.

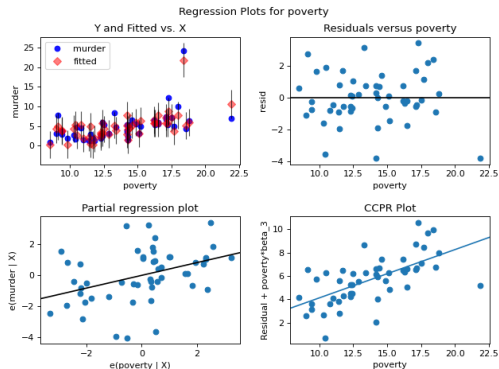


scikit-learn
algorithm cheat-sheet



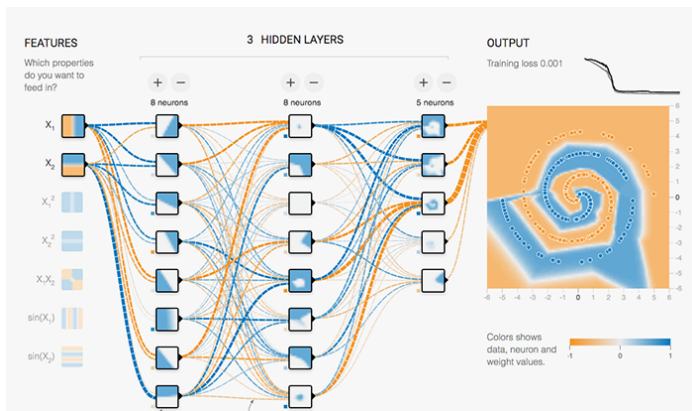
8. Statsmodels

- Modelos estadísticos tradicionales: OLS, ARIMA, etc.
- Análisis exploratorio de datos y pruebas estadísticas.
- Complemento perfecto para Scikit-learn.



9. TensorFlow

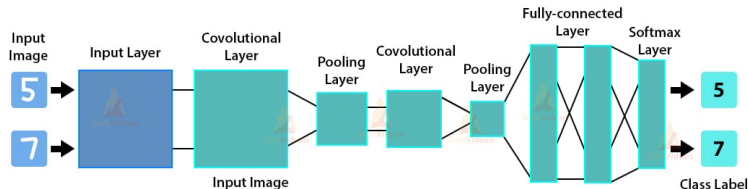
- Framework de Deep Learning de Google.
- Computación numérica eficiente con GPUs/TPUs.
- Para modelos complejos y redes neuronales.



- Interfaz de alto nivel para TensorFlow.
- Permite crear modelos de deep learning fácilmente.
- Ideal para principiantes y prototipado rápido.



Variety of Keras Layers

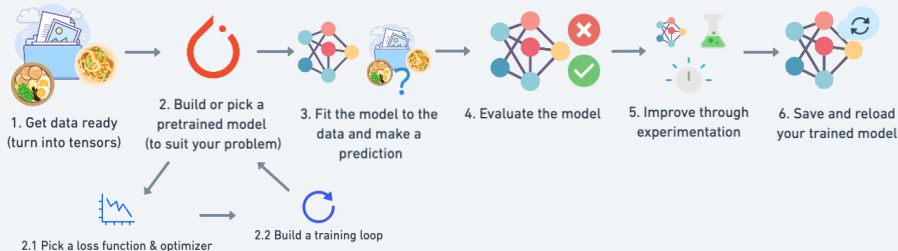


11. PyTorch

- Framework de deep learning de Facebook.
- Computación dinámica con fuerte integración a Python.
- Usado extensivamente en investigación y producción.

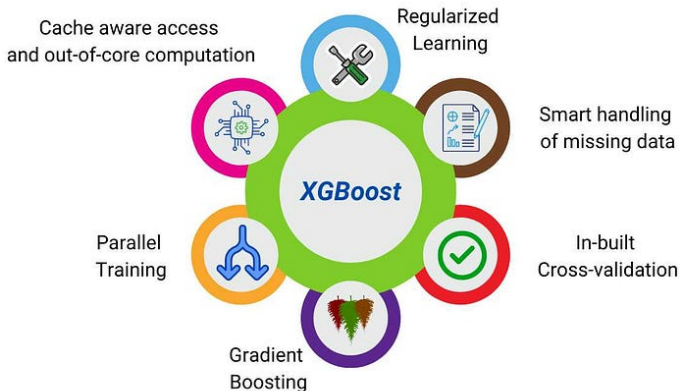


A PyTorch Workflow



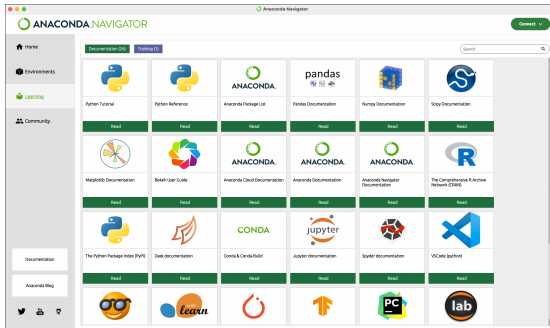
12. XGBoost

- Implementación optimizada de árboles de decisión.
- Muy eficiente y preciso para tareas de clasificación y regresión.
- Comúnmente utilizado en competencias de Kaggle.



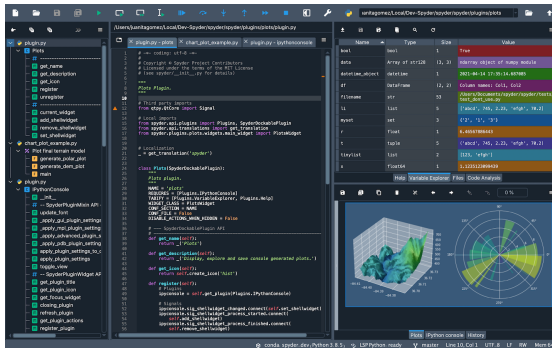
13. Anaconda

- Anaconda es una distribución gratuita y de código abierto de Python y R.
- Diseñada específicamente para ciencia de datos, análisis numérico, aprendizaje automático y computación científica.
- Incluye más de 1,500 paquetes preinstalados, así como herramientas como Jupyter Notebook, Spyder y Conda.
- Facilita la gestión de entornos virtuales y dependencias.



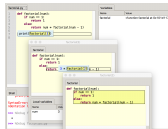
14.Spyder

- IDE (Entorno de Desarrollo Integrado) orientado a científicos e ingenieros.
- Integra consola interactiva, explorador de variables y editor avanzado.
- Perfecto para análisis numérico, visualización y programación estructurada en Python.
- Similar en estilo a MATLAB, lo que facilita la transición.



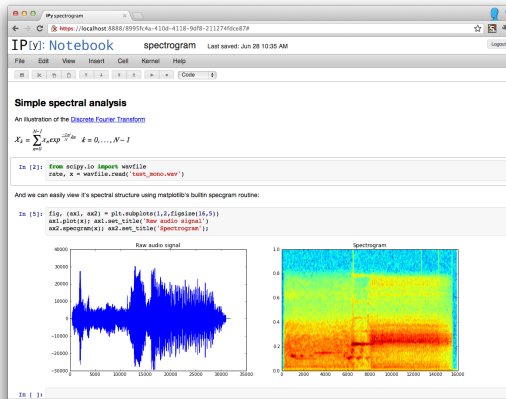
Thonny es un entorno de desarrollo integrado (IDE) gratuito y de código abierto diseñado específicamente para principiantes que están aprendiendo a programar en Python. Se caracteriza por su interfaz simple y sus herramientas visuales que facilitan la comprensión del código. Características Principales de Thonny:

- **Instalación Sencilla:** Thonny incluye su propio intérprete de Python, por lo que solo se necesita un instalador para empezar a usarlo, sin configuraciones complicadas.
 - **Interfaz Intuitiva:** La interfaz inicial es minimalista para evitar distracciones. Los usuarios pueden añadir funciones a medida que las necesitan.
 - **Depurador Integrado y Visual:** Permite ejecutar el código paso a paso, mostrando exactamente cómo Python evalúa las expresiones y cómo cambian las variables en tiempo real. Esto es fundamental para el aprendizaje.
 - **Ventana de Variables:** Una función útil es la capacidad de ver y rastrear todas las variables y cómo son afectadas por la ejecución del programa. Resaltado de Errores de Sintaxis: Ayuda a identificar errores comunes como paréntesis o comillas sin cerrar, facilitando la corrección del código.
 - **Soporte Multiplataforma:** Está disponible para Windows, macOS y Linux.
- Compatibilidad con MicroPython: Es una excelente opción para programar microcontroladores como ESP32 y Raspberry Pi Pico usando MicroPython.



15. Jupyter

- Entorno interactivo para escribir y ejecutar código Python.
- Permite combinar código, texto (Markdown), ecuaciones y visualizaciones.
- Muy usado en ciencia de datos, enseñanza y documentación de proyectos.
- Facilita la exploración de datos en tiempo real.



¿Qué es Pandas?

Pandas es una biblioteca de Python para el análisis y manipulación de datos estructurados. Ofrece herramientas poderosas como:

- DataFrames para manejar datos tabulares.
- Funciones de filtrado, agrupación y agregación.
- Soporte para datos faltantes y operaciones avanzadas.

Es ampliamente utilizado en ciencia de datos debido a su flexibilidad y eficiencia.

```
In [2]: 1 # Load our necessary libraries
        2 import pandas as pd
        3 import numpy as np
```

```
In [3]: 1 # Create data into CSV (that we'll import later on)
        2 # Let's say the data is for a veterinarian keeping tabs on his clients.
        3
        4 raw_data = {'pet_name': ['Woof', 'Chester', 'Rex', 'Mystery', 'Pumpkin'],
        5          'pet_last_name': ['Smith', 'Kim', '', 'Taylor', ''],
        6          'good_pet_score': [96, 34, 89, 92, 79],
        7          'type': ['dog', 'cat', 'mini-dinosaur', 'unknown', 'bird'],
        8          'amount_owed': ['5000', '9,000', '570, 622, 190']}
        9 df = pd.DataFrame(raw_data, columns = ['pet_name', 'pet_last_name', 'good_pet_score', 'type', 'amount_owed'])
       10 df
```

```
Out[3]:
```

	pet_name	pet_last_name	good_pet_score	type	amount_owed
0	Woof	Smith	96	dog	5000
1	Chester	Kim	34	cat	9,000
2	Rex		89	mini-dinosaur	570
3	Mystery	Taylor	92	unknown	622
4	Pumpkin		79	bird	190

Data

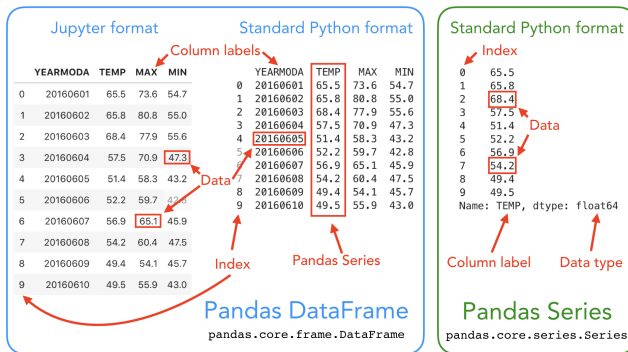
Column labels				
	name	city	age	py-score
101	Xavier	Mexico City	41	88.0
102	Ann	Toronto	28	79.0
103	Jana	Prague	33	81.0
104	Yi	Shanghai	34	80.0
105	Robin	Manchester	38	68.0
106	Amal	Cairo	31	61.0
107	Nori	Osaka	37	84.0
Data				
Row labels				

```
1 data = {
2     'name': ['Xavier', 'Ann', 'Jana', 'Yi', 'Robin', 'Amal', 'Nori'],
3     'city': ['Mexico City', 'Toronto', 'Prague', 'Shanghai', 'Manchester', 'Cairo', 'Osaka'],
4     'age': [41, 28, 33, 34, 38, 31, 37],
5     'py-score': [88.0, 79.0, 81.0, 80.0, 68.0, 61.0, 84.0]
6 }
7
8 row_labels = [101, 102, 103, 104, 105, 106, 107]
```

¿Qué es un DataFrame?

Un **DataFrame** es una estructura de datos bidimensional en Pandas que permite almacenar y manipular datos tabulares. Es ampliamente utilizado en ciencia de datos debido a:

- Su capacidad para manejar datos estructurados.
- Funciones integradas para limpiar, transformar y analizar datos.
- Soporte para múltiples formatos de entrada/salida (CSV, Excel, texto, etc.).
- Los DataFrames de Pandas son herramientas poderosas para manipular datos estructurados.
- Permite importar, limpiar, ordenar, agregar y exportar datos de manera eficiente.
- Es ideal para proyectos de ciencia de datos que involucren análisis y visualización.



Dataframe

```
1 import pandas as pd
2 # Data
3 data = {
4     'name': ['Xavier', 'Ann', 'Jana', 'Yi', 'Robin',
5             'Amal', 'Nori'],
6     'city': ['Mexico City', 'Toronto', 'Prague', 'Shanghai',
7             'Manchester', 'Cairo', 'Osaka'],
8     'age': [41, 28, 33, 34, 38, 31, 37],
9     'py-score': [88.0, 79.0, 81.0, 80.0, 68.0, 61.0, 84.0]
10 }
11 # etiquetas filas
12 row_labels = [101, 102, 103, 104, 105, 106, 107]
13 # Dataframe
14 df = pd.DataFrame(data=data, index=row_labels)
15 # imprimir dataframe
16 print(df)
17 print("\n")
18 # imprimir las dos primeras filas
19 print(df.head(n=2))
20 print("\n")
21 # imprimir las ultima dos filas
22 print(df.tail(n=2))
```

\$ python example_12_pandas_dataframe.py

	name	city	age	py-score
101	Xavier	Mexico City	41	88.0
102	Ann	Toronto	28	79.0
103	Jana	Prague	33	81.0
104	Yi	Shanghai	34	80.0
105	Robin	Manchester	38	68.0
106	Amal	Cairo	31	61.0
107	Nori	Osaka	37	84.0

	name	city	age	py-score
101	Xavier	Mexico City	41	88.0
102	Ann	Toronto	28	79.0

	name	city	age	py-score
106	Amal	Cairo	31	61.0
107	Nori	Osaka	37	84.0

■ Importar Archivo CSV

```
1 import pandas as pd
2
3 # Leer archivo CSV
4 df = pd.read_csv('data.csv')
5
6 # Mostrar las primeras filas
7 print(df.head(3))
```

Importar archivo CSV

```
$ python example_13_pandas_open_data_csv.py
```

		name	city	age	py-score
0	101	Xavier	Mexico City	41	88
1	102	Ann	Toronto	28	79
2	103	Jana	Prague	33	81

■ Importar Archivo de Texto

```
1 import pandas as pd
2
3 # Leer archivo de texto separado
  por una tabulacion
4 df = pd.read_csv('data.txt',
  delimiter = '\t')
5
6 # Mostrar las primeras filas
7 print(df.head(3))
```

Importar archivo de texto

```
$ python example_13_pandas_open_data_txt.py
```

		name	city	age	py-score
0	101	Xavier	Mexico City	41	88.0
1	102	Ann	Toronto	28	79.0
2	103	Jana	Prague	33	81.0

■ Importar Archivo Excel

```
1 import pandas as pd
2
3 # Leer archivo Excel
4 df = pd.read_excel('data.xlsx',
5                     sheet_name='data')
6
7 # Mostrar las primeras filas
8 print(df.head(3))
```

```
$ python example_13_pandas_open_data_xlsx.py
```

		name	city	age	py-score
0	101	Xavier	Mexico City	41	88
1	102	Ann	Toronto	28	79
2	103	Jana	Prague	33	81

Importar archivo Excel

■ Agregar Filas

```
1 import pandas as pd
2
3 # Crear DataFrame
4 data = {'Nombre': ['Ana', 'Juan'],
5         'Edad': [25, 30]}
6 df = pd.DataFrame(data)
7
8 # Agregar nueva fila
9 new_row = {'Nombre': 'Maria', 'Edad': 22}
10 df = df.append(new_row, ignore_index=True)
11
12 print(df)
```

```
$ python example_14_pandas_append_row.py
```

	Nombre	Edad
0	Ana	25
1	Juan	30

	Nombre	Edad
0	Ana	25
1	Juan	30
2	Maria	22

Agregar filas

Eliminar Filas y Rellenar Valores Faltantes

■ Eliminar Filas con Valores Faltantes

```
1 import pandas as pd
2
3 # Crear DataFrame con valores faltantes
4 data = {'Nombre': ['Ana', 'Juan', None],
5         'Edad': [25, None, 30]}
6 df = pd.DataFrame(data)
7
8 # Eliminar filas con valores faltantes
9 df_clean = df.dropna()
10
11 print(df_clean)
```

```
$ python example_14_pandas_clean_row.py
```

	Nombre	Edad
0	Ana	25.0

Eliminar filas con valores faltantes

■ Rellenar Valores Faltantes

```
1 import pandas as pd
2
3 # Crear DataFrame con valores faltantes
4 data = {'Nombre': ['Ana', 'Juan', None],
5         'Edad': [25, None, 30]}
6 df = pd.DataFrame(data)
7
8 # Rellenar valores faltantes
9 df_filled = df.fillna({'Nombre': 'Desconocido', 'Edad': 0})
10
11 print(df_filled)
```

```
$ python example_14_pandas_fill_missing_values.py
```

	Nombre	Edad
0	Ana	25.0
1	Juan	0.0
2	Desconocido	30.0

Rellenar valores faltantes

Ordenar y Agregar Datos

■ Ordenar Datos por Columna

```
1 import pandas as pd
2
3 # Crear DataFrame
4 data = {'Nombre': ['Ana', 'Juan', 'Maria'],
5         'Edad': [25, 30, 22]}
6 df = pd.DataFrame(data)
7
8 # Ordenar por edad
9 df_sorted = df.sort_values(by='Edad', ascending=
    True)
10
11 print(df_sorted)
```

```
$ python example_14_pandas_sort_values.py
Nombre Edad
2 Maria  22
0 Ana    25
1 Juan   30
```

Ordenar datos por columna

■ Agregar Nuevas Columnas

```
1 import pandas as pd
2
3 # Crear DataFrame
4 data = {'Nombre': ['Ana', 'Juan', 'Maria'],
5         'Edad': [25, 30, 22]}
6 df = pd.DataFrame(data)
7
8 # Agregar nueva columna
9 df['Ciudad'] = ['Lima', 'Bogota', 'Santiago']
10
11 print(df)
```

```
$ python example_14_pandas_add_column_values.py
Nombre Edad  Ciudad
0 Ana    25    Lima
1 Juan   30  Bogota
2 Maria  22  Santiago
```

Agregar nuevas columnas

Exportar DataFrame a CSV y DataFrame a Excel

■ Exportar DataFrame a CSV

```
1 import pandas as pd
2
3 # Crear DataFrame
4 data = {'Nombre': ['Ana', 'Juan'],
5         'Edad': [25, 30]}
6 df = pd.DataFrame(data)
7
8 # Exportar a CSV
9 df.to_csv('datos_exportados.csv', index=False)
```

```
$ python example_15_pandas_export_csv.py
$ ll datos_exportados.csv
-rw-rw-r-- 1 henry henry 27 Apr  3 21:25 datos_exportados.csv
```

Exportar DataFrame a CSV

■ Exportar DataFrame a Excel

```
1 import pandas as pd
2
3 # Crear DataFrame
4 data = {'Nombre': ['Ana', 'Juan'],
5         'Edad': [25, 30]}
6 df = pd.DataFrame(data)
7
8 # Exportar a Excel
9 df.to_excel('datos_exportados.xlsx', index=False)
```

```
$ python example_15_pandas_export_excel.py
$ ll datos_exportados.xlsx
-rw-rw-r-- 1 henry henry 4949 Apr  3 21:27 datos_exportados.xlsx
```

Exportar DataFrame a Excel

Análisis de Datos con Pandas

Pandas es una biblioteca poderosa para manipular y analizar datos estructurados.

```
1 import pandas as pd
2
3 # Crear un DataFrame
4 data = {
5     'Nombre': ['Ana', 'Juan', 'Maria', 'Jose',
6               'Roberto'],
7     'Edad': [25, 30, 22, 23, 33],
8     'Ciudad': ['Lima', 'Bogota', 'Santiago', 'Caracas', 'Bogota']
9 }
10 df = pd.DataFrame(data)
11
12 # Mostrar el DataFrame
13 print("DataFrame:")
14 print(df)
15
16 # Filtrar personas mayores de 24 años
17 filtro = df[df['Edad'] > 24]
18 print("\nPersonas mayores de 24 años:")
19 print(filtro)
```

\$ python example_1_pandas_Dataframe.py

DataFrame:

	Nombre	Edad	Ciudad
0	Ana	25	Lima
1	Juan	30	Bogota
2	Maria	22	Santiago
3	Jose	23	Caracas
4	Roberto	33	Bogota

Personas mayores de 24 años:

	Nombre	Edad	Ciudad
0	Ana	25	Lima
1	Juan	30	Bogota
4	Roberto	33	Bogota

Análisis básico con Pandas

Física - Cálculo de Velocidad Media y Clima - Análisis de Temperaturas

```
1
2 # Datos de tiempo y posicion
3 data = {'Tiempo (s)': [0, 1, 2, 3, 4],
4         'Posicion (m)': [0, 5, 10, 15, 20]}
5
6 df = pd.DataFrame(data)
7
8 # Calcular velocidad media
9 df['Velocidad Media (m/s)'] = df['Posicion (m)']
10    .diff() / df['Tiempo (s)'].diff()
11 print(df)
```

Velocidad media en física

```
$ python example_2_pandas_Dataframe_Fisica.py
```

Tiempo (s)	Posicion (m)	Velocidad Media (m/s)
0	0	NaN
1	5	5.0
2	10	5.0
3	15	5.0
4	20	5.0

```
1 import pandas as pd
2
3 # Datos de temperatura
4 data = {
5     'Ciudad': ['Lima', 'Bogota', 'Santiago'],
6     'Temperatura Promedio (C)': [18, 14, 12],
7     'Precipitacion Promedio (mm)': [20, 100, 40]
8 }
9 df = pd.DataFrame(data)
10
11 # Ordenar por temperatura
12 ordenado = df.sort_values(by='Temperatura Promedio (C)',
13                             ascending=False)
14 print(ordenado)
```

```
$ python example_6_pandas_Dataframe_Temperatura.py
```

Ciudad	Temperatura Promedio (C)	Precipitacion Promedio (mm)
0 Lima	18	20
1 Bogota	14	100
2 Santiago	12	40

Epidemiología - Tasa de Infectados y Salud - Análisis de Pacientes

```
1 import pandas as pd
2 # Datos de infectados
3 data = {
4     'País': ['Peru', 'Brasil', 'Mexico'],
5     'Poblacion': [33e6, 213e6, 128e6],
6     'Infectados': [1e6, 30e6, 5e6]
7 }
8 df = pd.DataFrame(data)
9
10 # Calcular tasa de infectados
11 df['Tasa Infectados (%)'] = (df['Infectados']/df
12     ['Poblacion']) * 100
13
14 print(df)
```

Tasa de infectados

```
$ python example_7_pandas_Dataframe_Epidemiologia.py
   País  Poblacion  Infectados  Tasa Infectados (%)
0  Peru   33000000.0   1000000.0         3.030303
1 Brasil 213000000.0  30000000.0        14.084507
2 Mexico 128000000.0   5000000.0         3.906250
```

```
1 import pandas as pd
2
3 # Datos de pacientes
4 data = {
5     'Paciente': ['P1', 'P2', 'P3'],
6     'Edad': [45, 32, 58],
7     'Presion Arterial': [120, 130, 140],
8     'Diabetes': ['No', 'Si', 'No']
9 }
10 df = pd.DataFrame(data)
11
12 # Filtrar pacientes con diabetes
13 filtro = df[df['Diabetes'] == 'Si']
14 print(filtro)
```

```
$ python example_11_pandas_Dataframe_Salud.py
   Paciente  Edad  Presion Arterial  Diabetes
1         P2    32             130         Si
```

Análisis de pacientes

Biología - Análisis de Especie y Bioinformática - Procesamiento de Secuencias Genéticas

```
1 import pandas as pd
2 # Datos de especies
3 data = {
4     'Especie': ['Leon', 'Tigre', 'Oso', 'Lobo'],
5     'Poblacion': [1200, 3500, 20000, 15000],
6     'Habitat': ['Sabana', 'Selva', 'Bosque', 'Tundra']
7 }
8 df = pd.DataFrame(data)
9
10 # Filtrar especies con poblacion > 5000
11 filtro = df[df['Poblacion'] > 5000]
12 print(filtro)
```

Distribución de especies

```
$ python example_4_pandas_Dataframe_Biologi.py
```

Especie	Poblacion	Habitat
2	Oso	20000 Bosque
3	Lobo	15000 Tundra

```
1 import pandas as pd
2
3 # Datos de genes
4 data = {
5     'Gen': ['GEN1', 'GEN2', 'GEN3'],
6     'Secuencia': ['ATGCTA', 'GCTAGC', 'TTAGGG'],
7     'Longitud': [6, 6, 6]
8 }
9 df = pd.DataFrame(data)
10
11 # Filtrar genes con longitud especifica
12 filtro = df[df['Longitud'] == 6]
13 print(filtro)
```

```
$ python example_5_pandas_Dataframe_Bioinformatica.py
```

	Gen	Secuencia	Longitud
0	GEN1	ATGCTA	6
1	GEN2	GCTAGC	6
2	GEN3	TTAGGG	6

Ventas - Análisis de Productos y Economía - Análisis de PIB

```
1 import pandas as pd
2
3 # Datos de ventas
4 data = {
5     'Producto': ['A', 'B', 'C', 'D'],
6     'Ventas': [100, 200, 150, 300],
7     'Costo Unitario': [10, 15, 20, 25]
8 }
9 df = pd.DataFrame(data)
10
11 # Calcular ingresos totales
12 df['Ingresos Totales'] = df['Ventas'] * df['
    Costo Unitario']
13 print(df)
```

```
$ python example_8_pandas_Dataframe_Ventas.py
```

	Producto	Ventas	Costo Unitario	Ingresos Totales
0	A	100	10	1000
1	B	200	15	3000
2	C	150	20	3000
3	D	300	25	7500

Análisis de productos más vendidos

```
1 import pandas as pd
2
3 # Datos de PIB
4 data = {
5     'Pais': ['Peru', 'Chile', 'Colombia'],
6     'PIB (USD)': [2.7e11, 3.1e11, 3.5e11],
7     'Poblacion': [33e6, 19e6, 51e6]
8 }
9 df = pd.DataFrame(data)
10
11 # Calcular PIB per capita
12 df['PIB Per Capita'] = df['PIB (USD)']/df['
    Poblacion']
13 print(df)
```

```
$ python example_9_pandas_Dataframe_Analisis_PBI.py
```

	Pais	PIB (USD)	Poblacion	PIB Per Capita
0	Peru	2.700000e+11	33000000.0	8181.818182
1	Chile	3.100000e+11	19000000.0	16315.789474
2	Colombia	3.500000e+11	51000000.0	6862.745098

Análisis de PIB por país

```
1 import pandas as pd
2
3 # Datos de calificaciones
4 data = {
5     'Estudiante': ['Ana', 'Juan', 'Maria'],
6     'Matematicas': [85, 90, 78],
7     'Fisica': [88, 92, 81]
8 }
9 df = pd.DataFrame(data)
10
11 # Calcular promedio por estudiante
12 df['Promedio'] = df[['Matematicas', 'Fisica']
13                    ].mean(axis=1)
14 print(df)
```

\$ python example_10_pandas_Dataframe_Educacion.py

	Estudiante	Matematicas	Fisica	Promedio
0	Ana	85	88	86.5
1	Juan	90	92	91.0
2	Maria	78	81	79.5

Análisis de calificaciones

Resumen:

- Pandas es una herramienta versátil para el análisis y manipulación de datos estructurados.
- Los ejemplos mostrados demuestran su aplicabilidad en diversos campos como física, biología, economía, salud, etc.
- Facilita tareas como filtrado, cálculos, ordenación y visualización de datos.

Recomendaciones:

- Familiarícese con las funciones básicas de Pandas (DataFrames, Series, filtros, etc.).
- Explore bibliotecas complementarias como NumPy, Matplotlib y Scikit-learn.
- Practique con conjuntos de datos reales para mejorar sus habilidades.

¡Pandas es esencial para cualquier proyecto de ciencia de datos!

¿Qué es Matplotlib?

Matplotlib es una biblioteca de Python para crear gráficos y visualizaciones de alta calidad. Es ampliamente utilizada en ciencia de datos debido a:

- Matplotlib es una herramienta poderosa para la visualización de datos en ciencia de datos.
- Su flexibilidad para crear gráficos personalizados.
- Soporte para múltiples tipos de gráficos (líneas, barras, dispersión, etc.).
- Integración con Pandas y NumPy para análisis de datos.

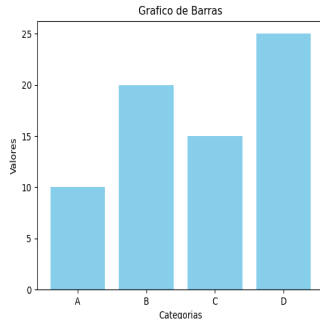
Es ideal para explorar y comunicar resultados de manera visual.

Ejemplo: Visualización con Matplotlib

Matplotlib es una biblioteca popular para crear gráficos en Python.

```
1 import matplotlib.pyplot as plt
2
3 # Datos
4 categorias = ['A', 'B', 'C', 'D']
5 valores = [10, 20, 15, 25]
6
7 # Crear grafico de barras
8 plt.bar(categorias, valores, color='skyblue'
9         )
10 plt.title('Grafico de Barras')
11 plt.xlabel('Categorias')
12 plt.ylabel('Valores')
13 plt.show()
```

Gráfico de barras con Matplotlib



Visualización con Matplotlib - Calculo de Velocidad Media

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Datos de tiempo y posicion
5 data = {'Tiempo (s)': [0, 1, 2, 3, 4],
6         'Posicion (m)': [0, 5, 10, 15, 20]}
7
8 df = pd.DataFrame(data)
9
10 # Calcular velocidad media
11 df['Velocidad Media (m/s)'] = df['Posicion (m)'].diff() / df['Tiempo (s)'].diff()
12 print(df)
13
14 posicion = df['Posicion (m)']
15 tiempo = df['Tiempo (s)']
16 velocidad = df['Velocidad Media (m/s)']
17
18 # Crear grafico
19 plt.plot(tiempo, posicion, marker='o', color='b')
20 plt.title('Posicion vs Tiempo')
21 plt.xlabel('Tiempo (s)')
22 plt.ylabel('Posicion (m)')
23 plt.grid(True)
24 plt.show()
```

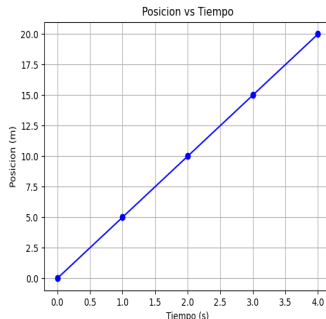
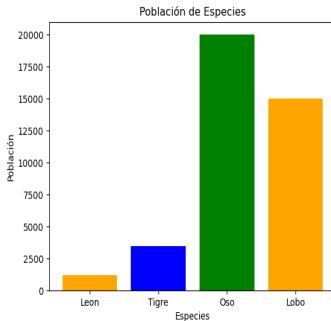


Gráfico de barras con Matplotlib

Visualización con Matplotlib - Análisis de Especies

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Datos de especies
5 data = {
6     'Especie': ['Leon', 'Tigre', 'Oso', 'Lobo'],
7     'Poblacion': [1200, 3500, 20000, 15000],
8     'Habitat': ['Sabana', 'Selva', 'Bosque', 'Tundra']
9 }
10 df = pd.DataFrame(data)
11
12 # Filtrar especies con poblacion > 5000
13 filtro = df[df['Poblacion'] > 5000]
14 print(filtro)
15
16 especies = df['Especie']
17 poblacion = df['Poblacion']
18
19 # Crear grafico de barras
20 plt.bar(especies, poblacion, color=['orange',
21     'blue', 'green'])
22 plt.title('Poblacion de Especies')
23 plt.xlabel('Especies')
24 plt.ylabel('Poblacion')
25 plt.show()
```

Gráfico de barras con Matplotlib



Visualización con Matplotlib - Epidemiología

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Datos de infectados
5 data = {
6     'País': ['Peru', 'Brasil', 'Mexico'],
7     'Poblacion': [33e6, 213e6, 128e6],
8     'Infectados': [1e6, 30e6, 5e6]
9 }
10 df = pd.DataFrame(data)
11
12 # Calcular tasa de infectados
13 df['Tasa Infectados (%)'] = (df['Infectados']
14                               / df['Poblacion']) * 100
15
16 print(df)
17
18 paises = df['País']
19 tasa_infectados = df['Tasa Infectados (%)']
20 poblacion = df['Poblacion']
21
22 # Crear grafico de barras
23 plt.barh(paises, tasa_infectados, color='skyblue')
24
25 plt.title('Tasa de Infectados por País (%)')
26 plt.xlabel('Tasa de Infectados (%)')
27 plt.ylabel('País')
28 plt.show()
```

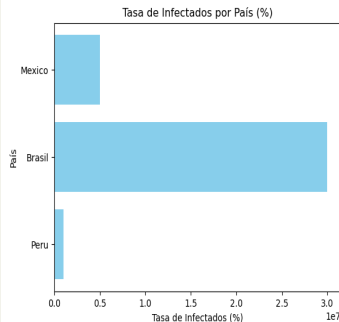


Gráfico de barras con Matplotlib

Visualización con Matplotlib - Educación

```
1 import pandas as pd
2 import matplotlib.pyplot as plt
3
4 # Datos de calificaciones
5 data = {
6     'Estudiante': ['Ana', 'Juan', 'Maria'],
7     'Matematicas': [85, 90, 78],
8     'Fisica': [88, 92, 81]
9 }
10 df = pd.DataFrame(data)
11
12 # Calcular promedio por estudiante
13 df['Promedio'] = df[['Matematicas', 'Fisica']
14                      ].mean(axis=1)
15 print(df)
16
17 asignaturas = df[['Matematicas', 'Fisica']]
18 promedios = df['Promedio']
19 estudiante = df['Estudiante']
20
21 # Crear grafico de lineas
22 plt.plot(estudiante, asignaturas, marker='D',
23          color='brown')
24 plt.title('Calificaciones Promedio por
25           Asignatura')
26 plt.xlabel('Asignatura')
27 plt.ylabel('Promedio')
28 plt.grid()
29 plt.show()
```

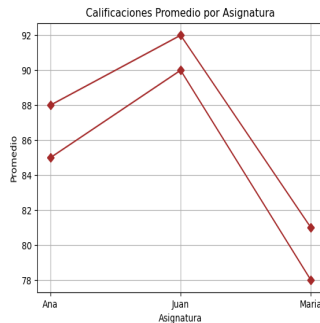


Gráfico de barras con Matplotlib

¿Qué es Seaborn?

Seaborn es una biblioteca de Python basada en Matplotlib que facilita la creación de gráficos estadísticos atractivos y profesionales. Es ampliamente utilizada en ciencia de datos debido a:

- Seaborn es una herramienta poderosa para la visualización de datos estadísticos en ciencia de datos.
- Su integración con Pandas para análisis de datos.
- Soporte para gráficos avanzados como mapas de calor, diagramas de caja, violín, etc.
- Estilos predefinidos que mejoran la estética de los gráficos.

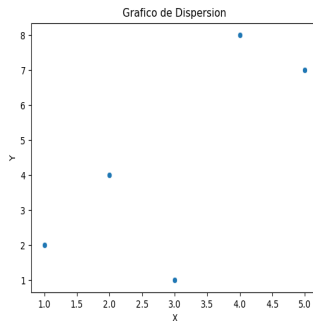
Es ideal para explorar y comunicar resultados complejos de manera visual.

Visualización con Seaborn

Seaborn es una biblioteca basada en Matplotlib que facilita la creación de gráficos estadísticos.

```
1 import seaborn as sns
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # Datos
6 data = pd.DataFrame({
7     'X': [1, 2, 3, 4, 5],
8     'Y': [2, 4, 1, 8, 7]
9 })
10
11 # Crear grafico de dispersion
12 sns.scatterplot(x='X', y='Y', data=data)
13 plt.title('Grafico de Dispersion')
14 plt.show()
```

Gráfico de dispersión con Seaborn



Visualización con Seaborn - Gráfico de Velocidad vs Tiempo

```
1 import seaborn as sns
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # Datos de tiempo y posicion
6 data = {'Tiempo (s)': [0, 1, 2, 3, 4],
7         'Posicion (m)': [0, 5, 10, 15, 20]}
8
9 df = pd.DataFrame(data)
10
11 # Calcular velocidad media
12 df['Velocidad Media (m/s)'] = df['Posicion (m)'].diff() / df['Tiempo (s)'].diff()
13 print(df)
14
15 posicion = df['Posicion (m)']
16 tiempo = df['Tiempo (s)']
17 velocidad = df['Velocidad Media (m/s)']
18
19 # Crear grafico
20 sns.regplot(x=tiempo, y=velocidad, ci=None, color='b')
21 plt.title('Relacion entre Velocidad y Tiempo')
22 plt.xlabel('Tiempo (s)')
23 plt.ylabel('Velocidad (m/s)')
24 plt.grid()
25 plt.show()
```

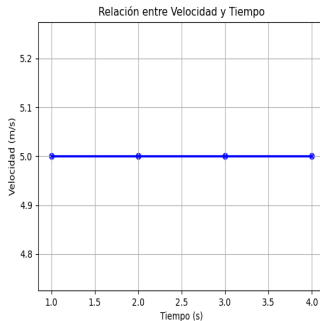
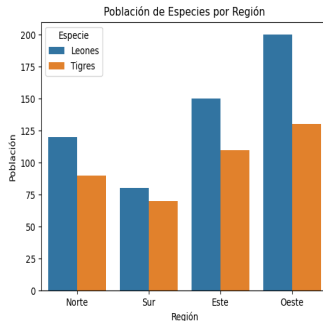


Gráfico de dispersión con Seaborn

Visualización con Seaborn - Población de Especies por Región

```
1 import seaborn as sns
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 # Datos
6 data = pd.DataFrame({
7     'Region': ['Norte', 'Sur', 'Este', 'Oeste'],
8     'Leones': [120, 80, 150, 200],
9     'Tigres': [90, 70, 110, 130]
10 })
11
12 # Crear grafico
13 data_melt = data.melt(id_vars='Region',
14                       var_name='Especie', value_name='Poblacion')
15
16 sns.barplot(data=data_melt, x='Region', y='Poblacion', hue='Especie')
17 plt.title('Poblacion de Especies por Region')
18 plt.show()
```

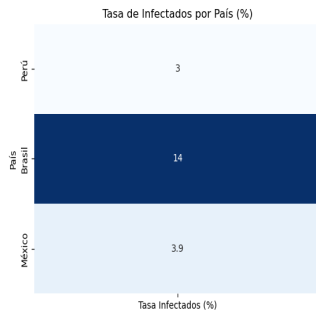
Gráfico de barras con Matplotlib



Visualización con Seaborn - Tasa de Infectados por País

```
1 import seaborn as sns
2 import matplotlib.pyplot as plt
3
4 # Datos
5 data = {
6     'País': ['Peru', 'Brasil', 'Mexico'],
7     'Tasa Infectados (%)': [3.0, 14.2, 3.9]
8 }
9 df = pd.DataFrame(data).set_index('País')
10
11 # Crear grafico
12 sns.heatmap(df, annot=True, cmap='Blues',
13             cbar=False)
14 plt.title('Tasa de Infectados por Pais (%)')
15 plt.show()
```

Gráfico de barras con Matplotlib



Visualización con Seaborn - PIB Per Cápita por País

```
1 import pandas as pd
2 import seaborn as sns
3 import matplotlib.pyplot as plt
4
5 # Datos
6 # Datos de PIB
7 data = {
8     'País': ['Peru', 'Chile', 'Colombia'],
9     'PIB (USD)': [2.7e11, 3.1e11, 3.5e11],
10    'Poblacion': [33e6, 19e6, 51e6]
11 }
12 df = pd.DataFrame(data)
13
14 # Calcular PIB per capita
15 df['PIB Per Capita'] = df['PIB (USD)'] / df[
    'Poblacion']
16
17 paises = df['País']
18 pib_per_capita = df['PIB (USD)']
19
20 # Crear grafico
21 sns.barplot(x=pib_per_capita, y=paises,
22             palette='viridis')
23 plt.title('PIB Per Capita por Pais (USD)')
24 plt.xlabel('PIB Per Capita (USD)')
25 plt.ylabel('País')
26 plt.show()
```

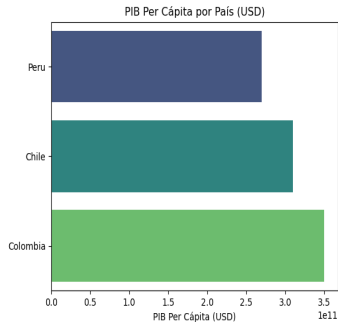


Gráfico de barras horizontales

¿Qué es Machine Learning?

El **machine learning** es una rama de la inteligencia artificial que permite a las computadoras aprender de los datos sin ser explícitamente programadas. En ciencia de datos, se utiliza para:

- Clasificación (por ejemplo, predecir categorías).
- Regresión (por ejemplo, predecir valores numéricos).
- Agrupamiento (por ejemplo, segmentar datos).
- Reducción de dimensionalidad.

En esta presentación, exploraremos ejemplos prácticos utilizando Python y bibliotecas como Scikit-learn, Keras, Matplotlib y Seaborn.

Regresión Lineal con Scikit-learn

Scikit-learn es una biblioteca ampliamente utilizada para machine learning.

```
1 from sklearn.linear_model import
    LinearRegression
2 import numpy as np
3
4 # Datos
5 X = np.array([[1], [2], [3], [4], [5]])
6 y = np.array([1.5, 3.1, 4.8, 6.2, 7.9])
7
8 # Crear modelo de regresion lineal
9 modelo = LinearRegression()
10 modelo.fit(X, y)
11
12 # Predecir
13 prediccion = modelo.predict([[6]])
14 print("Prediccion para X=6:", prediccion[0])
```

```
$ python example_1_sklearn_linear_regression.py
Prediccion para X=6: 9.47
```

Regresión lineal simple

Clasificación con Scikit-learn

Un ejemplo básico de clasificación usando Scikit-learn.

```
1 from sklearn.neighbors import
  KNeighborsClassifier
2 import numpy as np
3
4 # Datos
5 X = np.array([[1, 2], [2, 3], [3, 4], [6,
  7], [7, 8]])
6 y = np.array([0, 0, 0, 1, 1]) # Etiquetas (
  clases)
7
8 # Crear modelo KNN
9 modelo = KNeighborsClassifier(n_neighbors=3)
10 modelo.fit(X, y)
11
12 # Predecir
13 prediccion = modelo.predict([[5, 6]])
14 print("Clase predicha:", prediccion[0])
```

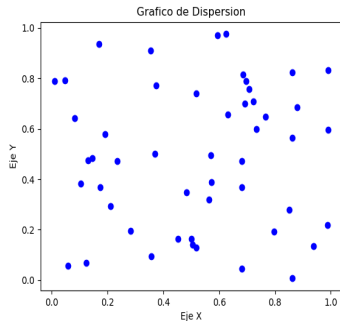
```
$ python example_2_sklearn_KNeighborsClassifier.py
Clase predicha: 1
```

Clasificación con K-Nearest Neighbors

Gráfico de Dispersión con Matplotlib

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3
4 # Datos
5 x = np.random.rand(50)
6 y = np.random.rand(50)
7
8 # Crear grafico
9 plt.scatter(x, y, color='blue')
10 plt.title('Grafico de Dispersion')
11 plt.xlabel('Eje X')
12 plt.ylabel('Eje Y')
13 plt.show()
```

Gráfico de dispersión



Evaluación de un Modelo de Clasificación

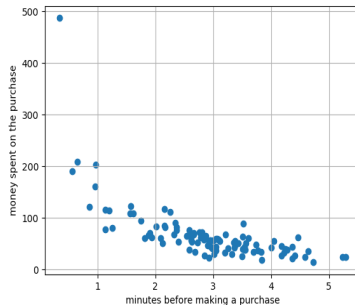
```
1 from sklearn.datasets import load_iris
2 from sklearn.model_selection import
  train_test_split
3 from sklearn.ensemble import
  RandomForestClassifier
4 from sklearn.metrics import accuracy_score
5
6 # Cargar datos
7 X, y = load_iris(return_X_y=True)
8
9 # Dividir datos
10 X_train, X_test, y_train, y_test =
    train_test_split(X, y, test_size=0.2)
11
12 # Entrenar modelo
13 modelo = RandomForestClassifier()
14 modelo.fit(X_train, y_train)
15
16 # Evaluar modelo
17 y_pred = modelo.predict(X_test)
18 precision = accuracy_score(y_test, y_pred)
19 print("Precision del modelo:", precision)
```

```
$ python example_6_sklearn_evaluacion.py
Precision del modelo: 0.9666666666666667
```

Evaluación de un modelo

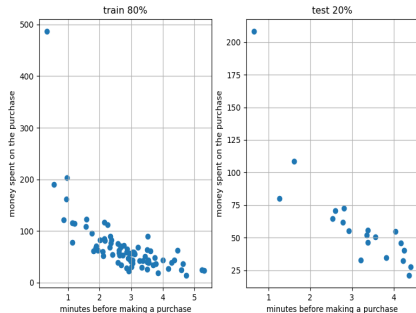
Machine Learning - Train/Test

```
1 import numpy
2 import matplotlib.pyplot as plt
3 numpy.random.seed(2)
4
5 x = numpy.random.normal(3, 1, 100)
6 y = numpy.random.normal(150, 40, 100) / x
7
8 plt.scatter(x, y)
9 plt.xlabel('minutes before making a purchase')
10 plt.ylabel('money spent on the purchase')
11 plt.show()
```



Train/Test - Datos

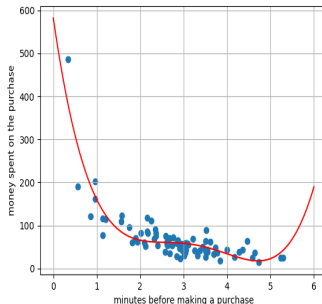
```
1 import numpy
2 import matplotlib.pyplot as plt
3 numpy.random.seed(2)
4
5 x = numpy.random.normal(3, 1, 100)
6 y = numpy.random.normal(150, 40, 100)
7   / x
8 # Separando la data train 80% y test
   20%
9 train_x = x[:80]
10 train_y = y[:80]
11
12 test_x = x[80:]
13 test_y = y[80:]
14
15 # Subplots
16 fig, (ax1, ax2) = plt.subplots(1, 2)
17 fig.suptitle('train 80% y test 20%')
18 # Display the Training Set
19 ax1.scatter(train_x, train_y)
20 ax2.grid()
21 # Display the Testing Set
22 ax2.scatter(test_x, test_y)
23 ax1.grid()
24 plt.show()
```



Train/Test - Datos

Mejor ajuste sería una regresión polinomial, así que tracemos una **línea de regresión polinomial**.

```
1 import numpy
2 import matplotlib.pyplot as plt
3 numpy.random.seed(2)
4
5 x = numpy.random.normal(3, 1, 100)
6 y = numpy.random.normal(150, 40, 100) / x
7
8 # Separando la data train 80% y test 20%
9 train_x = x[:80]
10 train_y = y[:80]
11
12 test_x = x[80:]
13 test_y = y[80:]
14
15 # fit the data set
16 mymodel = numpy.poly1d(numpy.polyfit(
17     train_x, train_y, 4))
18 myline = numpy.linspace(0, 6, 100)
19
20 plt.figure(2)
21 plt.scatter(train_x, train_y)
22 plt.plot(myline, mymodel(myline), 'r')
23 plt.xlabel('minutes before making a purchase')
24 plt.ylabel('money spent on the purchase')
25 plt.grid()
26 plt.show()
```



El resultado sugiere que el conjunto de datos se ajusta a una regresión polinómica, aunque podría darnos algunos resultados extraños si intentamos predecir valores fuera del conjunto de datos.

Observacion : la línea indica que un cliente que pase 6 minutos en la tienda haría una compra por un valor de 200. Eso probablemente sea una señal de sobreajuste (overfitting). **Qué pasa con el puntaje R-cuadrado:** El puntaje R-cuadrado es un buen indicador de qué tan bien se ajusta mi conjunto de datos al modelo.

Train/Test - Datos

```
1 import numpy
2 from sklearn.metrics import r2_score
3 numpy.random.seed(2)
4
5 x = numpy.random.normal(3, 1, 100)
6 y = numpy.random.normal(150, 40, 100) / x
7
8 # Separando la data train 80% y test 20%
9 train_x = x[:80]
10 train_y = y[:80]
11
12 test_x = x[80:]
13 test_y = y[80:]
14
15 # fit the data set
16 mymodel = numpy.poly1d(numpy.polyfit(
    train_x, train_y, 4))
17
18 # find the R2 score when using training
    data:
19 r2 = r2_score(train_y, mymodel(train_x))
20 print("R-squared - training data = %5.3f"%
    r2)
21
22 # find the R2 score when using testing data
    :
23 r2 = r2_score(test_y, mymodel(test_x))
24 print("R-squared - test data = %5.3f"% r2)
25
26 # How much money will a buying customer
    spend, if she or he stays in the shop
    for 5 minutes?
27 print("modelo = %5.3f"% mymodel(5))
```

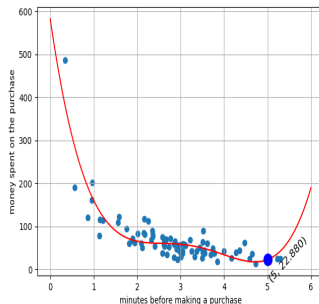
R-squared o R-cuadrado: Mide la relación entre el eje-x y el eje-y, y el valor varía de 0 a 1, donde 0 significa que no hay relación y 1 significa que están totalmente relacionados. El librería **sklearn** tiene un método llamado **r2_score()** que nos ayudará a encontrar esta relación.

- En este caso, nos gustaría medir la relación entre los minutos que un cliente permanece en la tienda y la cantidad de dinero que gasta. ajustan los datos de entrenamiento (**training data**) usando una regresión polinomial. El resultado 0.799 muestra que existe una relación aceptable.
- Encontrar el puntaje **r2_score()** usar los datos de prueba **test data**. El resultado 0.809 muestra que el modelo se ajusta bien al conjunto de prueba, y estamos seguros de que podemos usar el modelo para predecir valores futuros.
- ¿Cuánto dinero gastará un cliente comprador si permanece en la tienda durante 5 minutos?. El ejemplo predijo que el cliente gastaría 22.88 dólares, lo cual parece corresponder al diagrama

```
$ python example_10_models_predict_outcome_of_events_
Fit_the_Data_Set.py
R-squared - training data = 0.799
R-squared - test data = 0.809
modelo = 22.880
```

Train/Test - Datos

```
1 import numpy
2 import matplotlib.pyplot as plt
3 numpy.random.seed(2)
4
5 x = numpy.random.normal(3, 1, 100)
6 y = numpy.random.normal(150, 40, 100) / x
7
8 # Separando la data train 80% y test 20%
9 train_x = x[:80]
10 train_y = y[:80]
11
12 test_x = x[80:]
13 test_y = y[80:]
14
15 # fit the data set
16 mymodel = numpy.poly1d(numpy.polyfit(
17     train_x, train_y, 4))
18 myline = numpy.linspace(0, 6, 100)
19
20 # How much money will a buying customer
21 # spend, if she or he stays in the shop
22 # for 5 minutes?
23 print("modelo = %5.3f"% mymodel(5))
24
25 plt.figure(2)
26 plt.scatter(train_x, train_y)
27 plt.plot(myline, mymodel(myline), 'r')
28 plt.plot(5, mymodel(5), 'bo', markersize=12)
29 plt.text(5, mymodel(5), "(5, 22.880)",
30     rotation=40, fontsize=12,
31     verticalalignment='center')
32
33 plt.xlabel('minutes before making a purchase')
34 plt.ylabel('money spent on the purchase')
35 plt.grid()
36 plt.show()
```



```
$ python example_10_models_predict_outcome_of_events_
Fit_the_Data_Set.py
R-squared - training data = 0.799
R-squared - test data = 0.809
modelo = 22.880
```

Caso de estudio : ¿De qué color es la flor?

Imagina que estás estudiando un tipo de flor que puede ser de dos colores: azul o rojo. Has medido la longitud y el ancho de los pétalos de varias flores y quieres construir un modelo para predecir el color de una flor basándote en estas medidas.

Observa la data

	X1	X2	X3
0	44	91	1
1	99	16	0
2	48	2	0
3	56	98	1
4	47	45	0
5	37	89	1
6	35	24	0
7	28	81	1
8	46	2	0
9	68	32	0
10	75	40	0
11	52	44	0
12	61	37	0
13	69	85	1
14	46	16	0
15	37	14	0
16	65	52	0
17	37	93	1
18	16	65	0
19	71	25	0
20	91	16	0
21	37	100	1
22	9	15	0
23	61	76	0
24	26	91	1

Variables Predictoras

X1 = Ancho de petalo (mm)

X2 = Longitud del petalo (mm)

Variable Objetivo

X3 = Color de la flor (azul o rojo)



Arbol de decisiones de profundidad 3

```
1 import matplotlib.pyplot as plt
2 import pandas as pd
3
4 # Datos
5 data = {
6     'X1': [44, 99, 48, 56, 47, 37,
7           35, 28, 46, 68, 75, 52, 61, 69,
8           46, 37, 65, 37, 16, 71, 91, 37,
9           9, 61, 26],
10    'X2': [91, 16, 2, 98, 45, 89, 24,
11          81, 2, 32, 40, 44, 37, 85, 16,
12          14, 52, 93, 65, 25, 16, 100, 15,
13          76, 91],
14    'X3': [1, 0, 0, 1, 0, 1, 0, 1, 0,
15          0, 0, 0, 0, 1, 0, 0, 1, 0,
16          0, 0, 1, 0, 0, 1]
17 }
18 # Crear DataFrame
19 df = pd.DataFrame(data)
20
21 # Asignar colores basados en X3
22 colors = ['red' if x == 1 else 'blue'
23           for x in df['X3']]
24
25 # Crear grafico
26 plt.figure(figsize=(10, 6))
27 plt.scatter(df['X1'], df['X2'], c=
28             colors, s=100, alpha=0.7,
29             edgecolor='k')
30
31 # Etiquetas y titulo
32 plt.title('Ancho vs Longitud del
33           Petalo', fontsize=16)
34 plt.xlabel('Ancho del Petalo (mm)',
35            fontsize=14)
36 plt.ylabel('Longitud del Petalo (mm)',
37            fontsize=14)
```

```
1 # Leyenda
2 plt.scatter([], [], c='red', label=
3             'Flor Roja', s=100, edgecolor=
4             'k')
5 plt.scatter([], [], c='blue', label=
6             'Flor Azul', s=100, edgecolor=
7             'k')
8 plt.legend(fontsize=12)
9
10 # Mostrar grafico
11 plt.grid(True, linestyle='—',
12          alpha=0.5)
13 plt.tight_layout()
14 plt.show()
```

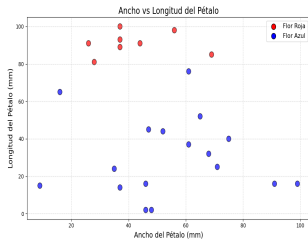
Variables Predictoras

X1 = Ancho de petalo (mm)

X2 = Longitud del petalo (mm)

Variable Objetivo

X3 = Color de la flor (azul o rojo)



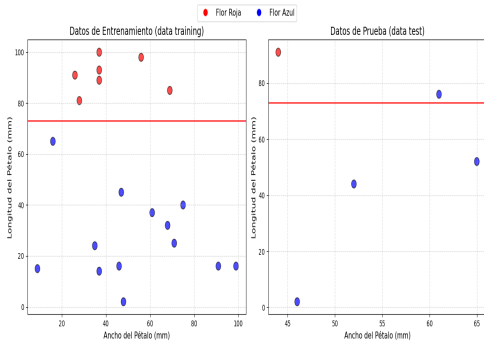
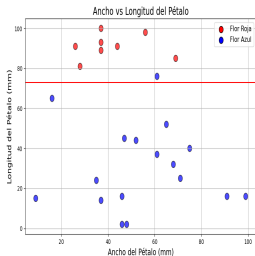
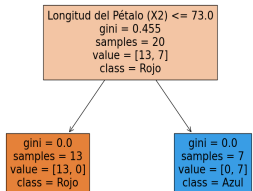
Arbol de decisiones de profundidad 3

```
1 # Importar bibliotecas necesarias
2 import pandas as pd
3 from sklearn.tree import
  DecisionTreeClassifier
4 from sklearn.model_selection import
  train_test_split
5 from sklearn.metrics import
  accuracy_score
6 from sklearn import tree
7 import matplotlib.pyplot as plt
8
9 # Datos
10 data = {
11     'X1': [44, 99, 48, 56, 47, 37, 35,
12           28, 46, 68, 75, 52, 61, 69, 46, 37,
13           65, 37, 16, 71, 91, 37, 9, 61, 26],
14     'X2': [91, 16, 2, 98, 45, 89, 24, 81,
15           2, 32, 40, 44, 37, 85, 16, 14, 52,
16           93, 65, 25, 16, 100, 15, 76, 91],
17     'X3': [1, 0, 0, 1, 0, 1, 0, 1, 0, 0,
18           0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 1,
19           0, 0, 1]
20 }
21 # Crear DataFrame
22 df = pd.DataFrame(data)
23
24 # Variables predictoras y objetivo
25 X = df[['X1', 'X2']] # Ancho y longitud
26                       # del petalo
27 y = df['X3']         # Color de la flor
28                       # (1 = azul, 0 = rojo)
29
30 # Dividir los datos en entrenamiento y
31 prueba
32 X_train, X_test, y_train, y_test =
33     train_test_split(X, y, test_size
34                     =0.2, random_state=42)
```

```
1 # Crear y entrenar el modelo de
  arbol de decision
2 modelo = DecisionTreeClassifier
  (max_depth=3, random_state
  =42)
3 modelo.fit(X_train, y_train)
4
5 # Realizar predicciones
6 y_pred = modelo.predict(X_test)
7
8 # Evaluar el modelo
9 precision = accuracy_score(
  y_test, y_pred)
10 print(f"Precision del modelo: {
  precision:.2f}")
11
12 # Visualizar el arbol de
  decision
13 plt.figure(figsize=(12, 8))
14 tree.plot_tree(modelo,
  feature_names=['Ancho del
  Petalo (X1)', 'Longitud
  del Petalo (X2)'],
  class_names=['
  Rojo', 'Azul'], filled=
  True)
15
16 plt.title("Arbol de Decision (
  Profundidad = 3)")
17 plt.show()
```

Arbol de decisiones de profundidad 3

Árbol de Decisión (Profundidad = 3)



- **Preprocesamiento de datos:** Limpieza y transformación son pasos clave antes de cualquier análisis.
- **Visualización:** Ayuda a comprender mejor los datos y comunicar resultados.
- **Modelos de ML:** Elija el modelo adecuado según el problema (regresión, clasificación, clustering, etc.).
- **Evaluar modelos:** Use métricas como precisión, recall o MSE para evaluar el rendimiento.

- Familiarícese con las principales bibliotecas (Pandas, NumPy, Matplotlib, Seaborn, Scikit-learn).
- Aprenda a manejar grandes volúmenes de datos con herramientas como Dask o PySpark.
- Documente su código y resultados para reproducibilidad.
- Explore frameworks avanzados como TensorFlow o PyTorch para deep learning.

Resumen:

- Python es una herramienta poderosa para ciencia de datos gracias a sus bibliotecas especializadas.
- Pandas facilita el análisis de datos estructurados.
- Matplotlib y Seaborn permiten crear visualizaciones claras y atractivas.
- Scikit-learn es ideal para implementar modelos de machine learning.

Recomendaciones:

- Practique con conjuntos de datos reales para mejorar sus habilidades.
- Manténgase actualizado con nuevas bibliotecas y técnicas.
- Colabore con otros profesionales en proyectos de ciencia de datos.

¡Python es una excelente elección para explorar y resolver problemas de ciencia de datos!

Markov Chain Monte Carlo (MCMC)

Several Python packages are available for performing Markov Chain Monte Carlo (MCMC) simulations. These packages offer various MCMC algorithms and tools for Bayesian parameter estimation and model fitting.

- **PyMC:** A probabilistic programming library that enables users to build Bayesian models with a simple Python API and fit them using MCMC and other sampling methods. It's known for its flexibility and integrates well with other scientific computing libraries.
- **emcee:** An MIT-licensed pure-Python implementation of Goodman & Weare's Affine Invariant Markov chain Monte Carlo (MCMC) Ensemble sampler. It is lightweight and particularly well-suited for high-dimensional problems.
- **pymcmcstat:** A Python implementation of the MATLAB toolbox "mcmcstat". It provides various Metropolis-based sampling techniques, including Metropolis-Hastings (MH), Adaptive-Metropolis (AM), Delayed-Rejection (DR), and Delayed Rejection Adaptive Metropolis (DRAM).
- **Sampyl:** A Python library implementing MCMC samplers designed for use in Bayesian parameter estimation. It allows users to define models using standard Python and packages like NumPy, and can automatically calculate gradients for samplers like NUTS using autograd.
- **PTMCMCSampler:** A parallel tempering MCMC sampler, useful for exploring complex posterior distributions and overcoming issues with local minima.
- **glabcmcmc:** A package implementing an Approximate Bayesian Computation Markov Chain Monte Carlo (ABC-MCMC) algorithm, combining global and local proposal strategies for enhanced sampling.

Markov Chain Monte Carlo (MCMC)

MCMC stands for Markov-Chain Monte Carlo, and is a method for fitting models to data.
Markov-Chain Monte Carlo

¿Preguntas?